

Comparative Study for Industrial Scheduling (JSSP): Exact Solvers vs. Reinforcement Learning

Performance analysis, reward modeling, and structural limits of dense architectures (MLP) regarding generalization.

Bob Lenglet, 06/14/2026

Github : <https://github.com/bobL59>

Abstract

This study compares an exact solver (OR-Tools) to an RL approach (MaskablePPO) on the JSSP FT10 instance. Although the RL model struggles to reach optimality (2608 t.u versus 930 t.u) due to the geometric limits of the MLP architecture, it enables real-time readjustment, paving the way for future implementations based on graphs (GNN).

1 : Introduction and Business Framework

1.1 The industrial challenge of the Job Shop Scheduling Problem (JSSP)

The Job Shop Scheduling Problem (JSSP) is an NP-hard optimization problem involving N orders (composed of several tasks) that must pass through M machines in a specific sequence. The objective is to complete all these orders in a minimum amount of time, which we call the Makespan.

In a factory, time is money. An idle machine or the formation of a bottleneck extends the total production time. Solving the JSSP while respecting the physical constraints of a factory therefore directly optimizes its profitability. This is the entire stake of this project.

1.2 Objective of the study: Reactivity vs. Optimality.

Faced with the JSSP, two approaches clash: the search for optimality or the need for reactivity.

On one hand, classic mathematical methods promise perfect results for a fixed, static factory, but they require significant computation times. On the other hand, the reality on the ground shows us that factories are almost never static. A machine can break down, an urgent order can be inserted, or plans can change at the last minute.

In these situations, we simply do not have the time for a solver to entirely recalculate a new solution, because every minute of downtime is highly expensive. This is why, in this study, I will evaluate whether an artificial intelligence (RL) is capable of learning empirical rules robust enough to readjust a schedule in real-time.

2 : The Deterministic Baseline: OR (Operations Research)

2.1 Mathematical modeling of the problem.

The mathematical modeling of the JSSP relies on a set of strict constraints that the solver must respect to guarantee a valid solution:

- A machine can only process one task at a time.
- The tasks of a single order have a strict processing sequence.
- In the context of the FT10 instance, each order (job) must imperatively pass through all the machines.
- Once a task has started on a machine, it cannot be interrupted or paused before its completion.
- All machines are available at time $t = 0$ and the transfer times from one machine to another are considered to be zero.

Now that the problem's constraints are established, we can solve it.

2.2 Resolution via Google OR-Tools and performance on the FT10 instance.

Google OR-Tools has established itself as an industry standard in optimization. It gathers several Operations Research algorithms, including constraint programming solvers that are particularly effective for the JSSP.

To evaluate the performance of our approach, this study will rely on the famous Fisher and Thompson 10x10 instance (named FT10). This historical configuration was specifically designed by the two mathematicians to offer a high level of complexity and to trap classic heuristics. Faced with this challenge, the exact solver finds the perfect solution: a Makespan of 930 time units (t.u). This score constitutes the insurmountable Makespan of our project. The Gantt chart below illustrates the meticulously precise organization of this optimal solution.

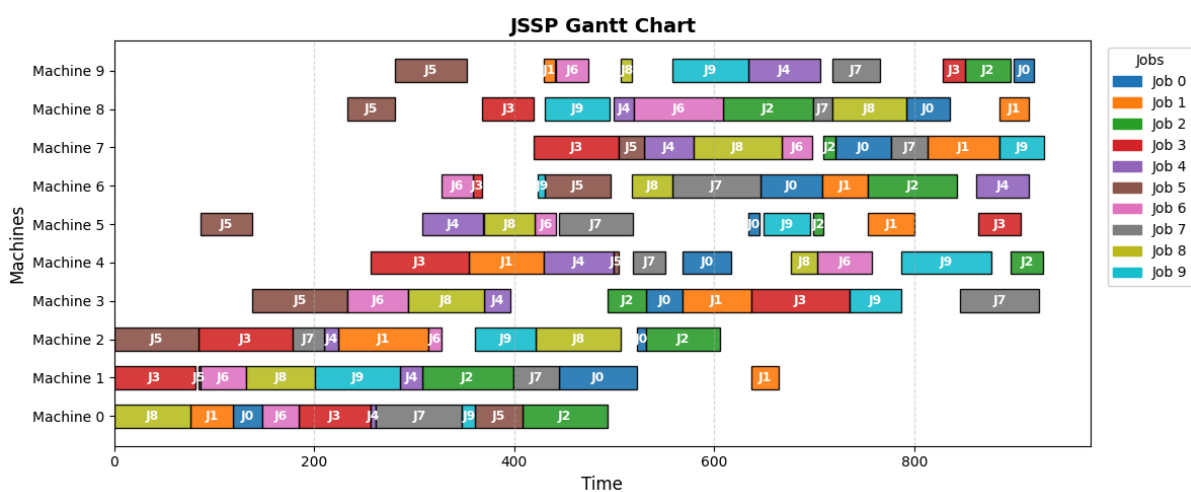


Figure 1: Gantt chart of the optimal solution (930 t.u) generated by OR-Tools on the FT10 instance.

3 : The Reinforcement Learning Approach

3.1 Why RL? The need for reactivity in a dynamic environment.

As previously explained, seeking the perfect solution with a mathematical solver requires a computation time that can quickly become a problem given the complexity of a factory. This is a delay we cannot afford in the field in the event of a breakdown or unforeseen event. This is where Reinforcement Learning comes in. The major advantage of AI is that almost all the heavy computational load is offloaded to the training phase. Once this phase is completed, the model acts as a simple engine capable of generating a valid schedule in a few milliseconds.

Therefore, to meet a need for reactivity, the RL approach accepts sacrificing optimality in favor of an instantaneous solution. The real challenge of this modeling is thus the following: succeeding in making this approximation sufficiently performant so that it becomes viable in production.

3.2 Environment modeling (Gymnasium and MaskablePPO).

For the AI environment, we will use the MLP (Multi Layer Perceptron) model; therefore, we will use a fixed-size, one-dimensional (1D) input vector. In this vector, the following will be stored:

- 1 - The availability times of each machine,
- 2 - The availability times of each part,
- 3 - An $N + 1$ anticipation: For each job, its duration and the required machine are retrieved. To keep values between 0 and 1, the durations are normalized (divided by the total time, also called the horizon) and the machine identifiers are encoded with One-Hot encoding.

At each time step t , the agent selects an action a_t corresponding to the identifier of the next order to be scheduled. The action space is therefore defined as a discrete space $A = \{0, 1, \dots, N - 1\}$, where N represents the total number of orders in the instance.

To accelerate training, we will use a Maskable PPO type learning algorithm. Consequently, illegal actions are impossible and therefore do not slow down the training.

The following table summarizes the hyperparameters and the structural configuration chosen for this training:

Parameter	Value / Configuration
Algorithm	Maskable PPO
Policy type	MlpPolicy
Network architecture	Actor: [256, 256] Critic: [256, 256]
Learning rate	3e-4
Entropy coefficient	0.00 or 0.05 depending on the scenarios (explained below)

Figure 2: Synthetic table of the hyperparameters and the structural configuration of the model

3.3 Implementation pitfalls: Naive Reward and Sparse Reward.

Our first idea for a reward method was the "Naive Reward". The idea was to punish the agent with the new end date of the task (-end_time). This approach quickly showed its limits because the agent placed short tasks as quickly as possible, creating massive bottlenecks in the future.

The second approach was obviously the "Sparse Reward". But since the agent only received a score at the end of the episode, it could not understand which specific action had caused it to fail.

Since these two basic reward approaches failed, we therefore had to adapt the reward ourselves for the remainder of the project.

4 : Optimization and Goal Alignment

4.1 Reward Shaping: Introduction of Delta Makespan to capture hidden time.

We thus thought of a different reward method: using the difference in Makespan between time T and time $T + 1$ (the Delta Makespan). This approach allows us to capture the concept of "hidden time". In this way, the agent is able to understand that it must parallelize tasks as much as possible to avoid increasing the overall time, and thus obtain a better reward.

```

makespan_before = max(self.machine_available_time)
#... Update according to the chosen action
makespan_after = max(self.machine_available_time)
reward = -float(makespan_after - makespan_before)

```

4.2 Convergence analysis.

Let us now observe the learning curves:

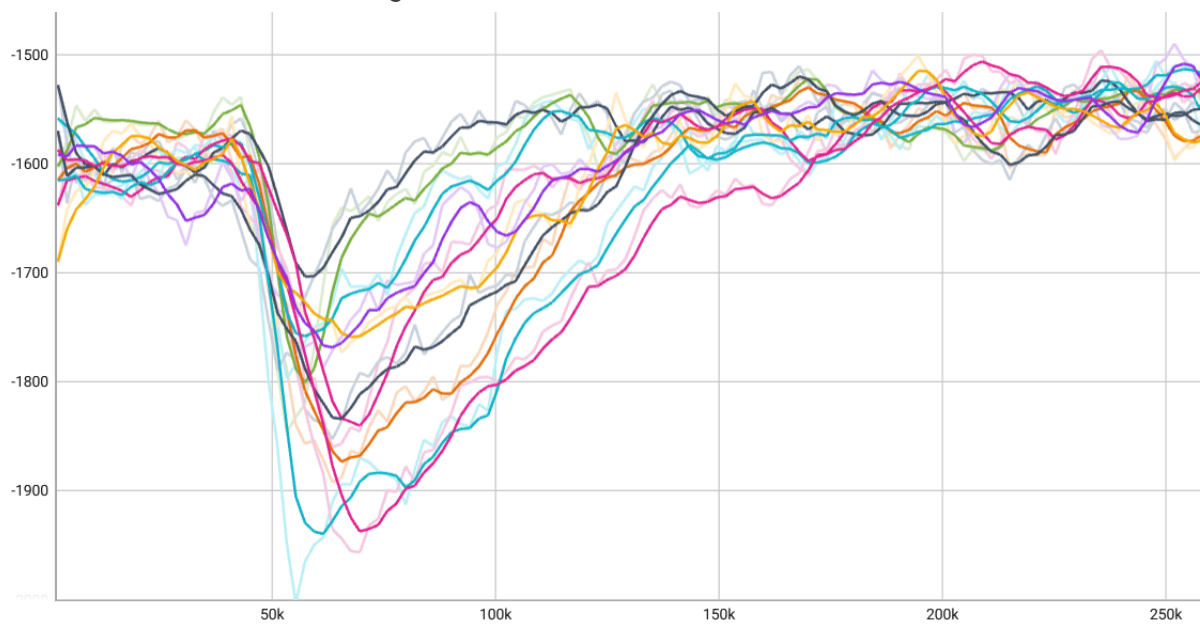


Figure 2: Evolution of the average reward (ep rew mean) over the 10 training sessions.

On these curves, we can distinguish a pattern that repeats itself in three phases:

- From 0 to ~60k steps: random exploration.
- From ~60k to ~175k steps: understanding of hidden time.
- From ~175k to 400k steps: reaching a local minimum (stagnation).

The reward on the y-axis is not the exact result on the FT10 instance, but it is a rather clear foretaste. We clearly notice that the models stagnate and do not approach the optimum of 930 t.u.

5 : Crash Test (FT10), Architectural Limits and Perspectives

5.1 Confrontation of the generalist agent with the FT10 reference instance and impact of entropy.

Our first attempt was inconclusive. The model stagnated at its local minimum. We therefore had an idea: if the model is stuck here, let's force it to explore more. With that came the idea of integrating entropy.

An initial test with an entropy of 0.05 proved conclusive (Makespan reduced by approximately 20 t.u on the FT10). However, this result could very well be the product of chance during training. This is why we launched a series of tests: 5 training sessions with zero entropy and 5 other training sessions with an entropy of 0.05. Here is the table of results on the FT10:

	Entropy	Makespan
Entraînement		
Entraînement 1	0.00	2666.0
Entraînement 2	0.00	2368.0
Entraînement 3	0.00	2619.0
Entraînement 4	0.00	2382.0
Entraînement 5	0.00	3113.0
Entraînement 6	0.05	2441.0
Entraînement 7	0.05	2667.0
Entraînement 8	0.05	2774.0
Entraînement 9	0.05	2732.0
Entraînement 10	0.05	2430.0

Figure 3: Table of results for the 10 models on the FT10 based on their entropy (Entraînement is "training" in french)

Coupled with these results, a calculation of the mean and standard deviation was performed:

The optimal solution of FT10 is **930 t.u**
Our model without entropy found a solution on average of **2629.60 +/- 302.02 t.u**
Our model with an entropy of 0.05 found a solution on average of **2608.80 +/- 162.77 t.u**

We notice that simply by changing one hyperparameter, we managed to reduce the overall time by about 20 t.u, but above all, the standard deviation was divided by almost 2. We therefore obtain a much more robust model thanks to its deeper exploration.

Regarding the inference time, we average at 0.68 ms. The promise of reactivity is fulfilled.

5.2 Stagnation analysis: The curse of dimensionality of the flat 1D vector.

The major problem with our models, and the primary cause of this stagnation, lies in the type of neural network used. Indeed, the "MLP" network is similar to a large rigid box. It takes a simple list of numbers but does not naturally understand dependency links (for example, that task A is physically blocked by task B). Without this relational logic, it fails to generalize its behavior when faced with the complex structure of the final instance, the FT10.

Furthermore, this architecture prevents us from implementing a progressive approach to difficulty (Curriculum Learning). We cannot start by giving it 3x3 problems, then 5x5, then 8×8 , to gently guide it towards the 10x10. Why? Because the input vector of an "MLP" network has a strictly fixed size. In our configuration, the agent is mathematically constrained and can only process one instance dimension at a time.

5.3 Perspectives: Towards disjunctive graph modeling and GNN architectures.

The logical evolution of this project would consist of replacing our 1D vector with a disjunctive graph structure, where nodes represent tasks and edges represent constraints (precedence and machines). This would require moving towards a GNN (Graph Neural Network) architecture. With such a network, the agent would finally be capable of:

- Naturally understanding the physical dependencies between tasks.
- Having a global and structured view of the entire factory.
- Being trained with progressive difficulty, as the size of a graph is not fixed at the input (Curriculum Learning).

Although such an implementation would require several months of research and development, this study has allowed me to accurately map the breaking point of classic models. By confronting the rigor of exact mathematics (OR) with the adaptability of Reinforcement Learning, this project illustrates my desire to tackle industrial challenges not with technological dogmas, but with an engineer's pragmatism focused on finding the most viable solution.

Github : <https://github.com/bobL59>